ACAS Xu input-region checker example

Input specification, counterexamples, and a verified correction

This example compares the input condition in Equation (7) with the bounds printed in Equation (8) of A General Framework for Property-Driven Machine Learning, version 2, Section 3.2. It executes the existing Lean reference checker and proves that a corrected translation preserves the source condition for every rational input.

The discrepancy is specific: Equation (7) limits the intruder's speed from above, while Equation (8) puts the same threshold in the lower-bound vector. The demonstration compares explicit encodings of these printed expressions. It does not execute an ACAS neural network or establish what was used in the paper's training experiments.

1. State, units, and demonstration domain

The state is the five-coordinate vector printed in the ACAS example:

```
x = (rho, theta, psi, v_own, v_intruder)
```

`State` is implemented as `Coord -> Rat`. Every value is an exact rational number. The two angles are in radians; distance is in feet and speeds are in feet per second.

Equation (8) leaves the outer bounds `l_min` and `u_max` symbolic. For this executable demonstration, we choose the following finite outer domain `D`:

Coordinate	Meaning	Lower	Upper
`rho`	Distance to the intruder, ft	0	60000
`theta`	Relative angular coordinate, rad	-1	1
`psi`	Relative angular coordinate, rad	-1	1
`v_own`	Own aircraft speed, ft/s	0	1200
`v_intruder`	Intruder speed, ft/s	0	1200

These bounds and the normalization below are explicit test choices. They are not presented as the complete domain or normalization of a deployed ACAS network. All comparisons use this same domain, so the observed disagreement comes from the direction of the intruder-speed bound.

In Lean, `domain x` means that every coordinate lies between these bounds. The independent source predicate is:

```
P7(x) = domain(x)
        AND rho >= 55947691/1000
        AND v_own >= 1145
        AND v_intruder <= 60
```

`55947691/1000` is exactly the decimal threshold `55947.691` from the paper.

2. The two translations

The source specification encodes `P7` as a formula. The reference compiler normalizes its thresholds and compiles the formula into a region of cells.

The candidate region transcribes the printed Equation (8) directly:

```
printed lower = (
  max(l_min.rho, 55947.691),
  l_min.theta,
  l_min.psi,
  max(l_min.v_own, 1145),
  max(l_min.v_intruder, 60)
)

printed upper = u_max
```

The default angular lower bounds above are the result of taking the maximum of `l_min` with the unconstrained entries in Equation (8). The representation does not require an infinity value in rational arithmetic.

For the chosen domain, these bounds mean:

```
P8(x) = domain(x)
        AND rho >= 55947691/1000
        AND v_own >= 1145
        AND v_intruder >= 60
```

The corrected translation leaves the intruder's lower bound at its domain minimum and restricts its upper bound:

```
corrected lower.v_intruder = l_min.v_intruder
corrected upper.v_intruder = min(u_max.v_intruder, 60)
```

The other coordinates retain their source constraints. Both translated regions are independently constructed boxes, rather than calls to the source compiler disguised as candidate translations.

3. Normalization

For every coordinate `i`, define:

```
offset[i] = lower[i]
scale[i]  = upper[i] - lower[i]

z[i] = (x[i] - offset[i]) / scale[i]
x[i] = z[i] * scale[i] + offset[i]
```

All five scales are strictly positive; Lean checks this obligation. The three source thresholds therefore become:

```
z_rho      >= 55947691/60000000
z_own      >= 229/240
z_intruder <= 1/20
```

The printed translation instead requires `z_intruder >= 1/20`.

```
Equation (7) formula ---- normalize + compile ---- referenceCheck
                                                        |
Physical point x ---- normalizePoint ---- z -------------+--- compare
                                                        |
Equation (8) bounds ---- normalizeRegion -------- printedCheck

Corrected bounds ------- normalizeRegion -------- correctedCheck
```

4. Concrete counterexamples

Fix `rho = 60000`, `theta = 0`, `psi = 0`, and `v_own = 1200`.

Intruder speed	Normalized speed	Equation (7)	Printed Equation (8)	Corrected bounds
50	`1/24`	`true`	`false`	`true`
60	`1/20`	`true`	`true`	`true`
100	`1/12`	`false`	`true`	`false`

The first row is a valid source input lost by the printed translation. The last row is an invalid source input admitted by that translation. Checking only the boundary at 60 would miss both problems.

For the first row, the complete states are:

```
physical x   = (60000, 0,   0,   1200, 50)
normalized z = (1,     1/2, 1/2, 1,    1/24)
```

The executable demonstration also checks values just below and above 60, the exact distance and own-speed thresholds, and violations of the outer domain. It contains 14 cases, with three Boolean decisions per case.

5. What Lean proves

`source_correct` connects the explicitly encoded formula to the independent predicate `P7`. `printed_region_correct` connects the printed box to `P8`. `corrected_region_correct` connects the corrected box to `P7`.

The key result is `corrected_agrees_everywhere`:

```
For every positive affine normalizer n
and every normalized rational point z:

    correctedCheck(n, z) = referenceCheck(n, z).
```

This is a universal proof for the specified domain and source condition; it is not inferred from the 14 runtime cases. It permits any positive affine normalization, not only the chosen demonstration normalization.

`loses_valid_input` and `admits_invalid_input` prove the first and last rows of the table in the Lean kernel. `printed_not_equivalent` proves that the printed translation and the reference checker are not equivalent. `boundary_agrees` and `outer_domain_retained` check additional boundaries.

The proof audit covers 18 roots, including the general compiler, normalization, and checker correctness theorems used by this example. The audit accepts only `propext`, `Classical.choice`, and `Quot.sound`; no

`sorry` or unchecked native-decision axiom is accepted.

The mapping from the paper's notation to the Lean definitions is explicit and inspectable. There is no automatic parser or proof of extraction from the PDF. The result concerns input membership; the advisory postcondition in Equation (9) and a network's behavior are outside this example.

6. Run the demonstration

Requirements: Python 3.11 or newer, and the repository's pinned Lean 4.35.0 and Mathlib installation with `lake` on `PATH`. The runner uses Python's standard library. No PyTorch installation or previous verification receipt is required for this example.

For an existing checkout, open a terminal in the repository root, the directory containing `README.md`, `lean/`, and `contribution/`. If the Lean dependencies are not yet available, first run:

```
cd lean
lake exe cache get
cd ..
```

Then run:

```
python3 -B contribution/spec_compiler/acas/verify.py
```

The runner builds the three compiler modules and `AcasExample.lean` in an isolated build directory, audits the proofs, executes the actual Lean checker, and compares its results with independent Python `Fraction` predicates. It prints the table and finishes with:

```
PASS: 18 audited roots; 14 cases; 42 checked decisions.
```

To additionally check a point of your choice:

```
python3 -B contribution/spec_compiler/acas/verify.py \
  --point 60000 0 0 1200 50
```

The argument order is `rho theta psi own_speed intruder_speed`. Decimal strings and fractions are interpreted exactly. For example:

```
python3 -B contribution/spec_compiler/acas/verify.py \
  --point 55947.691 0 0 1145 60
```

After a successful build, a shorter command runs the Lean checker directly without rebuilding or repeating the audit. Direct Lean input accepts integers or `n/d` fractions:

```
cd lean
LEAN_PATH=../contribution/.build/acas lake env lean --run \
  ../contribution/spec_compiler/acas/AcasMain.lean \
  60000 0 0 1200 50
cd ..
```

This returns JSON containing the physical point, normalized point, and the three decisions. With no point arguments, it returns all 14 cases.

7. Files and verification evidence

File	Purpose
`AcasExample.lean`	Coordinates, domain, predicates, candidate bounds, and proofs.
`AcasMain.lean`	Executable cases and exact rational input handling.
`Audit.lean`	Explicit requests for theorem axiom dependencies.
`verify.py`	Build, proof audit, independent execution checks, and display.
`../../results/acas_checker_verification.json`	Status, outcomes, theorem dependencies, and source/artifact hashes.

The runner also checks rejection of incomplete, duplicate, and untrusted proof-audit results, and rejects malformed command-line input. Generated objects and logs remain under the ignored `contribution/.build/acas/`.

The corrected translation is a proposed correction to the printed bounds. This example does not establish an additional bug in the authors' executable code or a change in their experimental results.